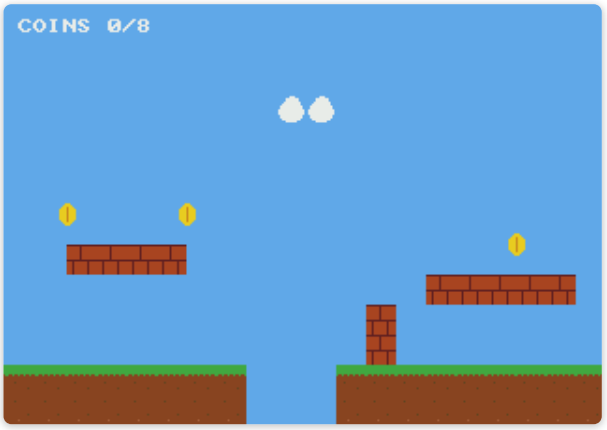
MEGABASIC 1.0

Reference Manual

The complete description of the MegaBASIC language, its graphics and sound systems, the compiler and the Mega Drive hardware behind them.



Star Catcher



Platform demo

Language · commands A-Z by topic · assets: PNG & Tiled · FM/PSG sound driver & MML · memory map · limits · compiler internals · complete example programs

Contents

1 Using the compiler
2 Language basics
3 Program structure
4 Decisions and loops
5 Variables, arrays and data
6 Text output
7 Timing
8 Graphics assets
9 Colours and palettes
10 Background planes and maps
11 Sprites
12 Joypad input
13 Maths
14 Memory access
15 Music and sound effects
16 Built-in constants
17 Hardware, memory and limits
18 How the compiler works
19 Example programs
20 Index

1 Using the compiler

mbc.py turns a MegaBASIC source file and its assets into a Mega Drive ROM. It needs only Python 3.8 or newer - no other packages.

Command line

```
python mbc.py SOURCE [-o OUTPUT.md] [--asm FILE.s] [--run EMULATOR] [-q]
```

Option	Meaning
`SOURCE`	The program (`.bas`). Asset file names inside it are relative to the source file's folder.
`-o FILE`	Output ROM name (default: source name with `.md`).
`--asm FILE`	Also write the complete generated 68000 assembly (runtime included) for study or debugging.
`--run EMU`	Start an emulator with the ROM after a successful build.
`-q`	Quiet: print only errors.

The build is a single step that takes a fraction of a second: the source is parsed, checked, translated to 68000 assembly, combined with the runtime library, the sound driver and the converted assets, assembled by MegaBASIC's own assembler and written as a ROM with a valid header and checksum. The ROM is padded to a multiple of 128 KB and runs on emulators, flash cartridges and real consoles (NTSC and PAL, with or without TMSS).

Errors are reported as `file:line: error: message` and the exit code is 1, so any editor or build system can pick them up. The VS Code extension does this automatically (see below).

VS Code extension

Feature	How
Build ROM	`Ctrl+Shift+B` or the package icon in the editor title
Build and run	`F5` (set `megabasic.emulatorPath` ; `megabasic.emulatorArgs` with `${rom}` for RetroArch etc.)
Help for a word	hover, or `F1` to open the reference panel
Completion	automatic, or `Ctrl+Space` ; after `asset.` for properties; after `@` in MML for instruments
Parameter hints	while typing arguments of commands, functions and your SUBs
Go to definition / outline	`F12` / Outline view: SUBs, FUNCTIONs, assets, songs, constants, labels
Errors on save	`megabasic.checkOnSave` (default on)
New project	Command palette: *MegaBASIC: New Game from Template*
Snippets	`game loop sub function if ifelse for while select music sfx instrument collide mapscroll ...`

Files with the extensions `.bas` and `.mbas` are treated as MegaBASIC. The compiler is bundled inside the extension; `megabasic.compilerPath` can point to another `mbc.py` .

2 Language basics

MegaBASIC is a structured, compiled BASIC. Line numbers are not used; programs are built from SUBs, FUNCTIONs and block statements.

Source text

- One statement per line; several statements can share a line separated by `:`. A line ending in `_` continues on the next line.
- Keywords and names are **not case sensitive**. Text in quotes keeps its case.
- Comments start with `'` or `REM`.
- Names: a letter or `_` followed by letters, digits, `_`. Names beginning with `__` are reserved for the library.
- Labels are a name followed by `:` alone at the start of a line (targets for GOTO and RESTORE).

Numbers

All values are **32-bit signed integers** (-2,147,483,648 to 2,147,483,647). Arithmetic wraps around on overflow. There are no floating-point numbers; use fixed point (e.g. positions in 1/16 pixel, angles 0..255, SIN/COS scaled by 256).

Literal	Example	Value
decimal	`1234`, `1_000_000`	1234, 1000000
hexadecimal	`$FF`, `&H1F`	255, 31
binary	`%1010`, `&B11`	10, 3
character	`"A"` (one character in quotes)	65
logical	`TRUE`, `FALSE`	-1, 0

Operators and precedence

From the highest to the lowest priority. Operators on the same row are evaluated left to right.

Priority	Operators	Notes
1	`( )`, function calls, array elements	
2	unary `-`, `+`	
3	`* / \ MOD << >> SHL SHR`	`/` and `\` are integer division truncating towards 0; `>>` is arithmetic (rounds down)
4	`+ -`	
5	`= <> < > <= >=`	result -1 (true) or 0 (false)
6	`NOT`	bitwise complement
7	`AND`	bitwise; short-circuit when both sides are conditions

8	`OR XOR`	bitwise; OR short-circuits on conditions

Examples

```
x = 2 + 3 * 4          ' 14
x = (2 + 3) * 4        ' 20
x = 1 << 4 + 1         ' 17  (<< binds like *)
ok = x > 5 AND x < 20  ' -1
bits = $F0 AND $3C     ' $30
IF (flags AND 4) <> 0 THEN ...
```

Watch out Because comparisons return -1, `NOT` works as a logical not on comparison results and TRUE/FALSE. On other numbers it is bitwise: `NOT 1` is -2, which is still "true". Store flags as TRUE/FALSE.

Variables and scope

- **Globals** are variables declared with `DIM` in the main program or assigned in the main program. They are visible everywhere and start at 0.
- **Locals** are parameters, variables declared with `LOCAL` / `DIM` inside a SUB/FUNCTION, and variables first assigned inside one that are not globals. They are created fresh (set to 0) on every call, so recursion works.
- Reading a name that is neither a variable, constant, asset nor function is a compile-time error - typos are caught early.
- **Arrays** are always global (an array DIMmed inside a SUB is shared by all calls of that SUB). Index bounds are checked at compile time for constant indexes only.
- Parameters are passed by value.

Statements at a glance

Group	Words
Program structure	`TITLE`, `REM`, `:`, `_`, `CONST`, `SUB`, `FUNCTION`, `RETURN`, `LOCAL`, `CALL`, `END`, `GOTO`
Decisions and loops	`EXIT`, `CONTINUE`, `IF`, `THEN`, `ELSE`, `ELSEIF`, `FOR`, `NEXT`, `TO`, `STEP`, `WHILE`, `WEND`, `DO`, `LOOP`, `UNTIL`, `REPEAT`, `SELECT`, `CASE`, `IS`
Variables, arrays and data	`DIM`, `AS`, `BYTE`, `WORD`, `LONG`, `LET`, `INC`, `DEC`, `SWAP`, `DATA`, `READ`, `RESTORE`, `AND`, `OR`, `XOR`, `NOT`, `MOD`, `SHL`, `SHR`
Text output	`PRINT`, `AT`, `CHR$`, `HEX$`, `PAD$`, `STR$`, `CLS`, `LOCATE`, `TEXTPAL`, `COLOR`, `TEXTPLANE`, `WINDOWROWS`
Timing	`VSYNC`, `WAIT`, `TIMER`
Graphics assets	`TILES`, `SPRITES`, `PALETTE`, `MAP`, `LOADTILES`, `LOADPAL`, `.BASE`, `.TILES`, `.FRAMES`, `.FTILES`, `.W / .H`, `.WIDTH / .HEIGHT`, `.TILESIZE`
Colours and palettes	`SETCOLOR`, `GETCOLOR`, `RGB`, `BACKDROP`, `BGCOLOR`, `BRIGHTNESS`, `FADEOUT`, `FADEIN`

Background planes and maps	`SETTILE`, `GETTILE`, `FILLTILES`, `CLEARPLANE`, `SCROLL`, `DRAWMAP`, `MAPSCROLL`, `MAPRESET`, `MAPTILE`, `MAPWIDTH`, `MAPHEIGHT`, `MAPTILESIZE`
Sprites	`SPRITE`, `SPRITESIZE`, `SPRITEPAL`, `SPRITEFLIP`, `SPRITEPRIORITY`, `SPRITEOFF`, `HIDESPRITE`, `CLEARSPRITES`, `COLLIDE`
Joypad input	`BUTTON`, `PRESSED`, `JOY`, `JOYNEW`
Maths	`ABS`, `SGN`, `MIN`, `MAX`, `CLAMP`, `RND`, `SIN`, `COS`, `ATAN2`, `SQR`, `RANDOMIZE`
Memory access	`PEEK`, `PEEKW`, `PEEKL`, `POKE`, `POKEW`, `POKEL`
Music and sound effects	`MUSIC`, `SFX`, `TEMPO`, `FM1`, `PSG1`, `NOISE`, `INSTRUMENT`, `ENVELOPE`, `PLAYMUSIC`, `STOPMUSIC`, `PAUSEMUSIC`, `RESUMEMUSIC`, `MUSICVOLUME`, `MUSICPLAYING`, `PLAYSFX`, `STOPSFX`
Built-in constants	`TRUE`, `FALSE`, `PLANE_A`, `PLANE_B`, `WINDOW`, `BTN_UP`, `BTN_DOWN`, `BTN_LEFT`, `BTN_RIGHT`, `BTN_B`, `BTN_C`, `BTN_A`, `BTN_START`, `BTN_Z`, `BTN_Y`, `BTN_X`, `BTN_MODE`, `HFLIP`, `VFLIP`, `PRIORITY`, `SCREEN_W`, `SCREEN_H`

3 Program structure

A program is a list of statements, executed from top to bottom, plus definitions (SUB, FUNCTION, CONST, assets, sound blocks) that may appear anywhere at the top level.

TITLE STATEMENT

```
TITLE "text"
```

Sets the game name stored in the ROM header (shown by emulators and flash carts). Up to 48 characters.

```
TITLE "SPACE BLASTER"
```

REM KEYWORD

```
REM comment   or   ' comment
```

A comment. Everything after REM or an apostrophe ' up to the end of the line is ignored by the compiler.

```
' This is a comment
REM so is this
x = 5   ' comments can follow a statement
```

: KEYWORD

```
statement : statement
```

Separates several statements written on one line.

```
x = 0 : y = 0 : VSYNC
```

_ KEYWORD

```
... _
```

An underscore at the very end of a line continues the statement on the next line.

```
IF COLLIDE(px, py, 16, 16, _
           ex, ey, 16, 16) THEN lives -= 1
```

CONST STATEMENT

```
CONST NAME = value [, NAME2 = value2 ...]
```

Defines a named constant. The value must be computable by the compiler (numbers, other constants, asset properties such as hero.FRAMES). Constants cost no RAM and make programs easier to read and change.

```
CONST SPEED = 3, MAXENEMY = 10
CONST CENTER = 320 / 2
```

SUB KEYWORD

```
SUB Name(param1, param2, ...)
    ...
END SUB
```

Defines a procedure: a named block of code that can be called with values (parameters). Parameters and variables created inside the SUB are local - they start at 0 on every call and do not affect the rest of the program. Variables declared with DIM in the main program are visible everywhere. Call a SUB by writing its name followed by the values, with or without parentheses, or with CALL.

```
SUB DrawBox(x, y, w)
    FOR i = 0 TO w - 1
        SETTILE PLANE_A, x + i, y, 1
    NEXT
END SUB

DrawBox 2, 3, 10
DrawBox(2, 5, 10)
CALL DrawBox(2, 7, 10)
```

FUNCTION KEYWORD

```
FUNCTION Name(param1, ...)
    ...
    RETURN value
END FUNCTION
```

Defines a function: like a SUB, but it gives back a value with RETURN (or by assigning to the function's own name). Functions can call themselves (recursion).

```
FUNCTION Distance(x1, y1, x2, y2)
    RETURN ABS(x2 - x1) + ABS(y2 - y1)
END FUNCTION

d = Distance(px, py, ex, ey)
```

RETURN STATEMENT

```
RETURN [value]
```

Leaves the current SUB or FUNCTION. Inside a FUNCTION, RETURN value gives the function's result. In the main program RETURN ends the program like END.

```
FUNCTION Sign(v)
    IF v < 0 THEN RETURN -1
    RETURN 1
END FUNCTION
```

LOCAL STATEMENT

```
LOCAL name [= value] [, name2 ...]
```

Declares variables that belong to the current SUB/FUNCTION only. Locals start at 0 on every call. (Any variable you assign inside a SUB that is not a global is automatically local, LOCAL just makes it explicit.)

```
SUB Explode(x, y)
    LOCAL i, frame = 6
    ...
END SUB
```

CALL STATEMENT

```
CALL Name(args)
```

Calls a SUB. Optional - writing just the name works too.

```
CALL ResetLevel(1)
```

END STATEMENT

```
END
```

Stops the program. The screen keeps showing what was drawn and music keeps playing.

```
IF lives = 0 THEN END
```

GOTO STATEMENT

```
GOTO label
```

Jumps to a label (a name followed by a colon at the start of a line) inside the same SUB/FUNCTION or the main program. Structured loops are usually clearer, but GOTO is there when you need it.

```
retry:
x = RND(320)
IF x < 16 THEN GOTO retry
```

4 Decisions and loops

Block statements can be nested freely. Indentation is not required but strongly recommended.

EXIT STATEMENT

```
EXIT FOR | EXIT WHILE | EXIT DO | EXIT REPEAT | EXIT SUB | EXIT FUNCTION
```

Leaves the innermost loop of the given kind, or the current SUB/FUNCTION, immediately.

```
FOR i = 0 TO 9
    IF enemyX(i) = 0 THEN EXIT FOR
NEXT
```

CONTINUE STATEMENT

```
CONTINUE FOR | CONTINUE WHILE | CONTINUE DO
```

Skips the rest of the loop body and starts the next round of the loop.

```
FOR i = 1 TO 10
    IF alive(i) = 0 THEN CONTINUE FOR
    MoveEnemy i
NEXT
```

IF KEYWORD

```
IF cond THEN statements [ELSE statements]

IF cond THEN
    ...
ELSEIF cond THEN
    ...
ELSE
    ...
END IF
```

Runs code only when a condition is true (not zero). The one-line form puts everything after THEN on the same line (several statements may be joined with ':'). The block form allows any number of ELSEIF parts and an optional ELSE. ENDIF is accepted as a synonym of END IF.

```
IF x > 300 THEN x = 300

IF lives = 0 THEN
    GameOver
ELSEIF lives = 1 THEN
    PRINT AT 0, 0, "LAST LIFE!"
ELSE
    PRINT AT 0, 0, "LIVES: "; lives
END IF
```

THEN KEYWORD

```
IF cond THEN ...
```

Part of IF.

ELSE KEYWORD

```
ELSE
```

Part of IF and SELECT CASE (CASE ELSE).

ELSEIF KEYWORD

```
ELSEIF cond THEN
```

An extra test inside a block IF.

FOR KEYWORD

```
FOR var = start TO end [STEP step]
    ...
NEXT [var]
```

Counts a variable from start to end (inclusive). STEP sets the amount added each round (default 1; can be negative). The end value is calculated once, before the loop starts.

```
FOR i = 1 TO 10
    PRINT i
NEXT

FOR y = 200 TO 0 STEP -8
    SPRITE 0, 100, y, 1
    VSYNC
NEXT y
```

NEXT KEYWORD

```
NEXT [var]
```

Ends a FOR loop.

TO KEYWORD

```
FOR ... TO ...  /  CASE a TO b
```

Part of FOR and SELECT CASE ranges.

STEP KEYWORD

```
FOR ... STEP n
```

Part of FOR.

WHILE KEYWORD

```
WHILE cond
    ...
WEND
```

Repeats the body as long as the condition is true. The test happens before each round, so the body may run zero times. END WHILE may be used instead of WEND.

```
WHILE y < 200
    y += 2
    VSYNC
WEND
```

WEND KEYWORD

```
WEND
```

Ends a WHILE loop.

DO KEYWORD

```
DO [WHILE cond | UNTIL cond]
    ...
LOOP [WHILE cond | UNTIL cond]
```

A flexible loop. With no condition it repeats forever (the usual game loop). The condition can be tested at the top (DO WHILE / DO UNTIL) or at the bottom (LOOP WHILE / LOOP UNTIL).

```
DO
    MovePlayer
    VSYNC
LOOP

DO
    VSYNC
LOOP UNTIL PRESSED(BTN_START)
```

LOOP KEYWORD

```
LOOP [WHILE cond | UNTIL cond]
```

Ends a DO loop.

UNTIL KEYWORD

```
UNTIL cond
```

Loop condition: stop when it becomes true.

REPEAT KEYWORD

```
REPEAT
    ...
UNTIL cond
```

Runs the body at least once and repeats until the condition becomes true.

```
REPEAT
    x = RND(320)
UNTIL ABS(x - px) > 64
```

SELECT KEYWORD

```
SELECT CASE value
    CASE 1
    CASE 2, 3
    CASE 4 TO 9
    CASE IS > 100
    CASE ELSE
END SELECT
```

Chooses one block of code depending on a value. A CASE can list several values, ranges (a TO b) and comparisons (IS > x, IS <= x ...). Only the first matching CASE runs.

```
SELECT CASE level
    CASE 1
        speed = 1
    CASE 2, 3
        speed = 2
    CASE 4 TO 8
        speed = 3
    CASE ELSE
        speed = 4
END SELECT
```

CASE KEYWORD

```
CASE values
```

Part of SELECT CASE.

IS KEYWORD

```
CASE IS > value
```

Comparison inside a CASE.

Worked example: SNAKE

Loops, SELECT CASE, 2-D BYTE arrays and SUBs in a complete classic game, drawn with text characters.

examples/reference/snake.bas

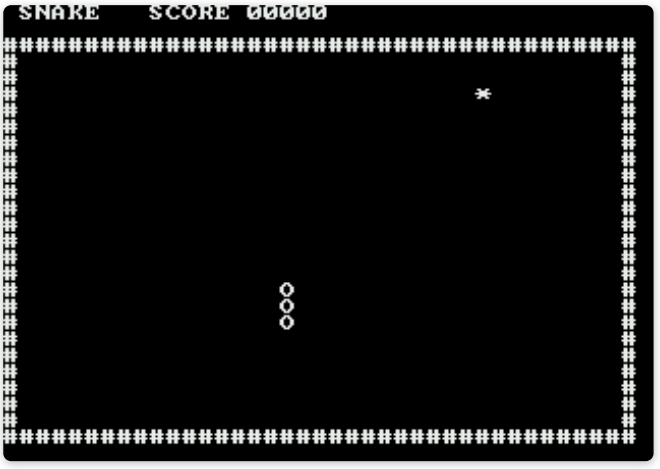
```
' SNAKE - arrays, DATA-free game logic, text graphics
TITLE "SNAKE"
CONST W = 38, H = 24, MAXLEN = 400
DIM bodyX(MAXLEN), bodyY(MAXLEN), grid(W, H) AS BYTE
DIM length, dir, foodX, foodY, score, speed, dead

SUB PlaceFood()
    DO
        foodX = 1 + RND(W - 1)
        foodY = 1 + RND(H - 1)
    LOOP UNTIL grid(foodX, foodY) = 0
    PRINT AT foodX, foodY + 2, "*"
END SUB

SUB NewGame()
    CLS
    FOR y = 0 TO H
        FOR x = 0 TO W
            grid(x, y) = 0
        NEXT
    NEXT
    FOR x = 0 TO W
        PRINT AT x, 2, "#" : PRINT AT x, H + 2, "#"
        grid(x, 0) = 1 : grid(x, H) = 1
    NEXT
    FOR y = 0 TO H
        PRINT AT 0, y + 2, "#" : PRINT AT W, y + 2, "#"
        grid(0, y) = 1 : grid(W, y) = 1
    NEXT
    length = 3 : dir = 1 : score = 0 : speed = 6 : dead = FALSE
    FOR i = 0 TO length - 1
        bodyX(i) = 10 - i : bodyY(i) = 10
        grid(bodyX(i), bodyY(i)) = 1
    NEXT
    PlaceFood
END SUB

SUB Step()
    LOCAL nx, ny
    nx = bodyX(0) : ny = bodyY(0)
    SELECT CASE dir
        CASE 0 : ny -= 1
        CASE 1 : nx += 1
        CASE 2 : ny += 1
        CASE 3 : nx -= 1
    END SELECT
    IF grid(nx, ny) THEN dead = TRUE : EXIT SUB
    IF nx = foodX AND ny = foodY THEN
        IF length < MAXLEN THEN length += 1
        score += 10
        IF speed > 2 AND score MOD 50 = 0 THEN speed -= 1
        PlaceFood
    ELSE
        PRINT AT bodyX(length - 1), bodyY(length - 1) + 2, " "
        grid(bodyX(length - 1), bodyY(length - 1)) = 0
    END IF
    FOR i = length - 1 TO 1 STEP -1
        bodyX(i) = bodyX(i - 1) : bodyY(i) = bodyY(i - 1)
    NEXT
```

```
60      bodyX(0) = nx : bodyY(0) = ny
61      grid(nx, ny) = 1
62      PRINT AT nx, ny + 2, "O"
63  END SUB
64
65  NewGame
66  DO
67      FOR t = 1 TO speed
68          IF BUTTON(BTN_UP) AND dir <> 2 THEN dir = 0
69          IF BUTTON(BTN_RIGHT) AND dir <> 3 THEN dir = 1
70          IF BUTTON(BTN_DOWN) AND dir <> 0 THEN dir = 2
71          IF BUTTON(BTN_LEFT) AND dir <> 1 THEN dir = 3
72          VSYNC
73      NEXT
74      Step
75      PRINT AT 1, 0, "SNAKE   SCORE "; PAD$(score, 5)
76      IF dead THEN
77          PRINT AT 14, 13, " GAME OVER "
78          WAIT 120
79          NewGame
80      END IF
81  LOOP
```



snake.bas

5 Variables, arrays and data

Variables, arrays, assignment shortcuts, DATA tables and the operators.

DIM STATEMENT

```
DIM name [= value]
DIM name(size) [AS BYTE | WORD | LONG] [= {v0, v1, ...}]
DIM name(w, h) [AS type]
```

Declares variables and arrays. All numbers are 32-bit whole numbers (-2147483648 to 2147483647). DIM a(10) creates 11 elements, a(0) to a(10). Arrays may have more dimensions. AS BYTE (0..255) and AS WORD (-32768..32767) arrays use less RAM. DIM in the main program makes variables visible to all SUBs and FUNCTIONs. Variables start at 0. Array indexes are not checked at run time - stay inside the bounds.

```
DIM score, lives = 3
DIM enemyX(9), enemyY(9)
DIM level(39, 27) AS BYTE
DIM speeds(3) = {1, 2, 4, 8}
```

AS KEYWORD

```
AS BYTE | WORD | LONG
```

Array element type in DIM.

BYTE KEYWORD

```
AS BYTE
```

Array element type: 8 bit, 0 to 255.

WORD KEYWORD

```
AS WORD
```

Array element type: 16 bit, -32768 to 32767.

LONG KEYWORD

```
AS LONG
```

Array element type: 32 bit (the default). INTEGER is a synonym.

LET STATEMENT

```
[LET] var = expression
```

Assignment. The word LET is optional. Compound forms +=, -=, *= and /= change a variable in place.

```
LET x = 5
x = x + 1
x += 1
score *= 2
```

INC STATEMENT

```
INC var [, amount]
```

Adds 1 (or amount) to a variable or array element. Very fast.

```
INC score, 10
INC frame
```

DEC STATEMENT

```
DEC var [, amount]
```

Subtracts 1 (or amount) from a variable or array element.

```
DEC lives
```

SWAP STATEMENT

```
SWAP a, b
```

Exchanges the values of two variables or array elements.

```
SWAP x1, x2
```

DATA STATEMENT

```
DATA value, value, ... | "text"
```

Stores constant numbers in the ROM to be read with READ. Text in quotes is stored as character codes followed by 0. Put a label before DATA lines so RESTORE can find them.

```
level1:
DATA 10, 20, 30, 40
names:
DATA "PLAYER ONE"
```

READ STATEMENT

```
READ var [, var ...]
```

Reads the next value(s) from DATA into variables or array elements.

```
RESTORE level1
FOR i = 0 TO 3
    READ enemyX(i)
NEXT
```

RESTORE STATEMENT

```
RESTORE [label]
```

Sets the DATA read position to the first DATA after label (or to the very first DATA).

```
RESTORE level2
```

AND OPERATOR

```
a AND b
```

Bitwise AND. With conditions (comparisons) it means 'both are true' and is evaluated with short-circuit.

```
IF x > 0 AND x < 320 THEN visible = TRUE
low = value AND 15
```

OR OPERATOR

```
a OR b
```

Bitwise OR / logical 'either is true'.

```
IF BUTTON(BTN_A OR BTN_B) THEN Jump
```

XOR OPERATOR

```
a XOR b
```

Bitwise exclusive OR.

```
flag = flag XOR 1
```

NOT OPERATOR

```
NOT a
```

Bitwise NOT. NOT TRUE = FALSE, NOT FALSE = TRUE.

```
IF NOT gameOver THEN Play
```

MOD OPERATOR

```
a MOD b
```

Remainder of a division (sign follows a).

```
IF frame MOD 60 = 0 THEN INC seconds
```

SHL OPERATOR

```
a SHL n  or  a << n
```

Shift bits left (multiply by 2^n, very fast).

```
x = 1 << 4    ' 16
```

SHR OPERATOR

```
a SHR n  or  a >> n
```

Arithmetic shift right (divide by 2^n rounding down, very fast).

```
px = subx >> 4
```

6 Text output

The built-in 8×8 font (ASCII 32-127) is loaded at start-up into VRAM tiles 1440-1535. Text cells are 8×8 pixels: 40 columns x 28 rows.

PRINT STATEMENT

```
PRINT [AT x, y,] item [; item ...] [;]
```

Writes text and numbers. Items are text in quotes, numbers/expressions, CHR$(code), HEX$(value [, digits]), PAD$(value, width) (zero padded) or STR$(value). Separate items with ; or ,. AT x, y positions the text (x 0..39, y 0..27 in 8×8 character cells). Without AT the text goes to the cursor and a new line follows unless the line ends with ;. Text uses the built-in font and colour 15 of the text palette (see TEXTPAL).

```
PRINT "HELLO"
PRINT AT 1, 1, "SCORE: "; PAD$(score, 6)
PRINT AT 30, 1, "HI "; HEX$(addr, 6)
PRINT "A"; "B";
```

AT KEYWORD

```
PRINT AT x, y, ...
```

Text position for PRINT.

CHR$ FUNCTION

```
CHR$(code)
```

Inside PRINT: prints the character with this ASCII code.

```
PRINT CHR$(65)   ' A
```

HEX$ FUNCTION

```
HEX$(value [, digits])
```

Inside PRINT: prints a number in hexadecimal (default 4 digits).

```
PRINT HEX$(255, 2)   ' FF
```

PAD$ FUNCTION

```
PAD$(value, width)
```

Inside PRINT: prints a number with leading zeros to the given width.

```
PRINT PAD$(42, 5)   ' 00042
```

STR$ FUNCTION

```
STR$(value)
```

Inside PRINT: prints a number (same as printing the value directly).

```
PRINT STR$(x)
```

CLS STATEMENT

```
CLS
```

Clears the text plane and moves the cursor to the top-left corner.

```
CLS
```

LOCATE STATEMENT

```
LOCATE x, y
```

Moves the text cursor to column x, row y.

```
LOCATE 10, 5
PRINT "HERE"
```

TEXTPAL STATEMENT

```
TEXTPAL palette
```

Chooses which palette (0-3) text uses. Text pixels use colour 15 of that palette. Palettes 2 and 3 start with a default set of colours (colour 15 = white). COLOR is a synonym.

```
TEXTPAL 3
SETCOLOR 3, 15, RGB(255, 255, 0)   ' yellow text
```

COLOR STATEMENT

```
COLOR palette
```

Same as TEXTPAL.

TEXTPLANE STATEMENT

```
TEXTPLANE PLANE_A | PLANE_B | WINDOW
```

Chooses the plane text is written to (default PLANE_A).

```
WINDOWROWS 2
TEXTPLANE WINDOW
PRINT AT 1, 0, "SCORE"
```

WINDOWROWS STATEMENT

```
WINDOWROWS rows
```

Shows the WINDOW plane over the top rows of the screen (0 = off). The window never scrolls, which makes it perfect for a status bar while both planes scroll.

```
WINDOWROWS 3
```

7 Timing

The console produces 60 frames per second (NTSC) or 50 (PAL). Everything in a game is timed in frames.

VSYNC STATEMENT

```
VSYNC
```

Waits for the next video frame (60 per second on NTSC, 50 on PAL) and then sends the sprite positions, scroll values and palette changes to the video chip. Every game loop should call VSYNC once.

```
DO
    MovePlayer
    VSYNC
LOOP
```

WAIT STATEMENT

```
WAIT frames
```

Waits the given number of frames (60 = one second on NTSC).

```
WAIT 120   ' two seconds
```

TIMER FUNCTION

```
TIMER
```

The number of frames since the console was switched on. Useful for animation and as a random seed.

```
frame = (TIMER >> 3) AND 3
RANDOMIZE TIMER
```

8 Graphics assets

Assets are converted when the program is compiled - the PNG and CSV files are not needed at run time.

Image conversion rules

- PNG of any type is accepted (indexed, grey, RGB, RGBA; 1-16 bit). Interlaced PNGs must be re-saved without interlacing.
- Each image may use at most **16 colours**, colour 0 being transparent. Colours are reduced to the Mega Drive's 3 bits per channel (512 colours).
- **Indexed PNG** with indices below 16: the indices and palette order are kept unchanged. Use this to share one palette between several images.
- **True-colour PNG**: pixels with alpha < 128 become colour 0. Without transparency, magenta #FF00FF is colour 0 if present, else the top-left pixel's colour. Other colours get indices in order of first appearance.
- Tiles are stored 4 bits per pixel, 32 bytes per 8×8 tile.

Tile order

Asset	Order of the 8×8 tiles
`TILES t = "x.png"`	left to right, top to bottom - tile k = Tiled tile ID k
`TILES t = "x.png", 16`	per 16×16 metatile (row-major): top-left, top-right, bottom-left, bottom-right. Metatile k occupies tiles 4k..4k+3
`SPRITES s = "x.png", w, h`	per frame, **column-major** (the order the VDP needs for multi-tile sprites). Frame f occupies tiles f*w*h ...

Tiled maps

Export the layer as CSV (*File > Export As > CSV*). The tileset must be a single image with no margin or spacing, its first GID 1. Horizontal/vertical flip flags are converted to VDP flip bits (diagonal flips are ignored). The map is stored twice in ROM: as ready-to-draw VDP cell words (for DRAWMAP / MAPSCROLL) and as Tiled IDs (for MAPTILE). Maps can be any size.

TILES STATEMENT

```
TILES name = "file.png" [, tilesize]
```

Imports a PNG as background tiles. tilesize is 8 (default) or 16/24/32 for 'metatiles' (each big tile is split into 8×8 pieces in left-to-right, top-to-bottom order). The tile order matches Tiled's tile IDs. The image may use at most 16 colours (colour 0 = transparent: alpha, magenta #FF00FF, or the top-left pixel of a PNG without transparency; indexed PNGs keep their own palette order).

```
TILES grass = "grass.png"
TILES world = "world16.png", 16
```

SPRITES STATEMENT

```
SPRITES name = "file.png", width, height
```

Imports a sprite sheet. The image is cut into frames of width x height tiles (1-4 each, i.e. 8 to 32 pixels), read left to right, top to bottom. Frames are stored in the column order the hardware needs, so a frame can be shown with SPRITE n, x, y, name, frame.

```
SPRITES hero = "hero.png", 2, 2    ' 16x16 frames
SPRITES boss = "boss.png", 4, 4    ' 32x32 frames
```

PALETTE STATEMENT

```
PALETTE name = "file.png"
```

Imports only the colours of a PNG (up to 16) for use with LOADPAL.

```
PALETTE night = "night_colours.png"
```

MAP STATEMENT

```
MAP name = "file.csv" [, tilesize]
```

Imports a map made with the Tiled editor (File > Export As > CSV, or a layer saved in CSV format). Tile IDs are Tiled's (empty = 0). Flipped tiles keep their flip. tilesize must match the TILES asset (8 or 16...).

```
MAP level1 = "level1.csv", 16
```

LOADTILES STATEMENT

```
LOADTILES asset [, vramTile]
```

Copies a TILES or SPRITES asset into video memory. Without vramTile the next free position is used automatically. The position is remembered in asset.BASE. Tiles 1 to 1439 are free for your graphics.

```
LOADTILES hero
LOADTILES grass, 100
PRINT hero.BASE
```

LOADPAL STATEMENT

```
LOADPAL asset, palette
```

Loads the 16 colours of an image asset into one of the 4 hardware palettes (0-3).

```
LOADPAL hero, 1
```

.BASE PROPERTY

```
asset.BASE
```

Video memory tile number where LOADTILES put the asset.

```
tile = grass.BASE + 3
```

.TILES PROPERTY

```
asset.TILES
```

Number of 8×8 tiles in the asset.

```
PRINT hero.TILES
```

.FRAMES PROPERTY

```
asset.FRAMES
```

Number of frames in a SPRITES asset (or metatiles in a TILES asset).

```
frame = (frame + 1) MOD hero.FRAMES
```

.FTILES PROPERTY

```
asset.FTILES
```

Tiles per frame (width x height).

.W / .H PROPERTY

```
asset.W, asset.H
```

Frame size in tiles.

.WIDTH / .HEIGHT PROPERTY

```
map.WIDTH, map.HEIGHT  /  image.WIDTH, image.HEIGHT
```

For a MAP: size in Tiled tiles. For an image: size in pixels.

```
camMax = level.WIDTH * 16 - 320
```

.TILESIZE PROPERTY

```
map.TILESIZE
```

Tile size of a MAP in pixels.

9 Colours and palettes

The VDP has 4 palettes of 16 colours (CRAM). Colour 0 of every palette is transparent. MegaBASIC keeps a copy of the palettes in RAM and uploads it during VSYNC, so changes never cause flicker.

SETCOLOR STATEMENT

```
SETCOLOR palette, index, colour
```

Changes one colour (index 0-15) of a palette (0-3). Colours are Mega Drive values, easiest made with RGB().

```
SETCOLOR 0, 0, RGB(0, 0, 80)
```

GETCOLOR FUNCTION

```
GETCOLOR(palette, index)
```

Returns a colour from a palette.

```
c = GETCOLOR(1, 5)
```

RGB FUNCTION

```
RGB(red, green, blue)
```

Makes a Mega Drive colour from red, green and blue values 0-255 (the console has 8 levels per channel).

```
SETCOLOR 3, 15, RGB(255, 200, 0)
```

BACKDROP STATEMENT

```
BACKDROP palette, index
```

Chooses the colour shown where nothing else is drawn. BGCOLOR is a synonym.

```
BACKDROP 0, 0
```

BGCOLOR STATEMENT

```
BGCOLOR palette, index
```

Same as BACKDROP.

BRIGHTNESS STATEMENT

```
BRIGHTNESS level
```

Sets the brightness of all palettes: 0 = black, 16 = normal.

```
BRIGHTNESS 8
```

FADEOUT STATEMENT

```
FADEOUT [speed]
```

Fades the screen to black. speed = frames per step (default 2).

```
FADEOUT
FADEOUT 4
```

FADEIN STATEMENT

```
FADEIN [speed]
```

Fades the screen in from black.

```
FADEIN 2
```

10 Background planes and maps

Plane A and plane B are 64 × 32 cells (512 × 256 pixels) and wrap around in both directions. The screen shows 40 × 28 cells of each.

SETTILE STATEMENT

```
SETTILE plane, x, y, tile [, palette [, flags]]
```

Puts one 8×8 tile on a background plane (64×32 cells, coordinates wrap around). flags can add HFLIP, VFLIP and PRIORITY.

```
SETTILE PLANE_B, 10, 5, grass.BASE, 0
SETTILE PLANE_A, 3, 3, 7, 1, HFLIP OR PRIORITY
```

GETTILE FUNCTION

```
GETTILE(plane, x, y)
```

Reads the tile value stored at a plane cell (tile number plus palette/flip bits).

```
t = GETTILE(PLANE_B, cx, cy) AND 2047
```

FILLTILES STATEMENT

```
FILLTILES plane, x, y, width, height, tile [, palette [, flags]]
```

Fills a rectangle of a plane with one tile.

```
FILLTILES PLANE_B, 0, 26, 40, 2, ground.BASE, 0
```

CLEARPLANE STATEMENT

```
CLEARPLANE plane
```

Clears a whole plane.

```
CLEARPLANE PLANE_B
```

SCROLL STATEMENT

```
SCROLL plane, x, y
```

Scrolls PLANE_A or PLANE_B so that pixel (x, y) of the plane is at the top-left of the screen. Planes are 512×256 pixels and wrap around. Takes effect at the next VSYNC.

```
SCROLL PLANE_B, camx / 2, 0   ' slow parallax
SCROLL PLANE_A, camx, 0
```

DRAWMAP STATEMENT

```
DRAWMAP map, plane, tiles, palette [, mapX, mapY [, planeX, planeY [, width, height]]]
```

Draws (part of) a Tiled map onto a plane. tiles is the TILES asset used by the map (or a VRAM tile number). By default it draws 64×32 cells starting at map cell (0,0). Coordinates are in 8×8 cells.

```
DRAWMAP level, PLANE_B, world, 0
```

MAPSCROLL STATEMENT

```
MAPSCROLL map, plane, tiles, palette, cameraX, cameraY
```

The easy way to scroll a big map: call it every frame with the camera position in pixels. It draws the map columns and rows that come into view and scrolls the plane. Maps may be any size.

```
camx = CLAMP(px - 160, 0, level.WIDTH * 16 - 320)
MAPSCROLL level, PLANE_B, world, 0, camx, 0
```

MAPRESET STATEMENT

```
MAPRESET plane
```

Makes the next MAPSCROLL on this plane redraw everything (use after changing level).

```
MAPRESET PLANE_B
```

MAPTILE FUNCTION

```
MAPTILE(map, tileX, tileY)
```

Returns the Tiled tile number (ID + 1, 0 = empty) at a map position in map tiles. Outside the map it returns 0. This is how you do collisions with the level.

```
IF MAPTILE(level, (px + 8) >> 4, (py + 16) >> 4) <> 0 THEN onGround = TRUE
```

MAPWIDTH FUNCTION

```
MAPWIDTH(map)
```

Width of a map in Tiled tiles.

MAPHEIGHT FUNCTION

```
MAPHEIGHT(map)
```

Height of a map in Tiled tiles.

MAPTILESIZE FUNCTION

```
MAPTILESIZE(map)
```

Tile size of a map in pixels.

11 Sprites

The VDP shows up to 80 sprites; each is 1-4 tiles wide and high. MegaBASIC keeps a sprite table in RAM and transfers it by DMA at every VSYNC.

Sprite size codes, link order and the hardware table are handled by the runtime. Positions are screen pixels; sprites more than 32 pixels outside the screen are hidden automatically. The VDP draws at most 20 sprites (320 sprite pixels) per scan line in 320-pixel mode.

SPRITE STATEMENT

```
SPRITE n, x, y, tile
SPRITE n, x, y, spriteAsset, frame
```

Shows hardware sprite n (0-79) at screen position x, y. The second form takes a frame of a SPRITES asset and sets the sprite size automatically. Sprites outside the screen are hidden. Lower numbers are drawn in front of higher numbers. Changes appear at the next VSYNC. At most 20 sprites can share one screen line.

```
SPRITE 0, px, py, hero, frame
SPRITE 5, 100, 50, 1
```

SPRITESIZE STATEMENT

```
SPRITESIZE n, width, height
```

Sets a sprite's size in tiles (1-4 each). Not needed with the asset form of SPRITE.

```
SPRITESIZE 3, 2, 2
```

SPRITEPAL STATEMENT

```
SPRITEPAL n, palette
```

Chooses the palette (0-3) of a sprite.

```
SPRITEPAL 0, 1
```

SPRITEFLIP STATEMENT

```
SPRITEFLIP n, hflip, vflip
```

Mirrors a sprite horizontally and/or vertically (TRUE/FALSE).

```
SPRITEFLIP 0, facingLeft, FALSE
```

SPRITEPRIORITY STATEMENT

```
SPRITEPRIORITY n, high
```

TRUE draws the sprite in front of high-priority background tiles.

```
SPRITEPRIORITY 0, TRUE
```

SPRITEOFF STATEMENT

```
SPRITEOFF n
```

Hides a sprite. HIDESPRITE is a synonym.

```
SPRITEOFF 4
```

HIDESPRITE STATEMENT

```
HIDESPRITE n
```

Same as SPRITEOFF.

CLEARSPRITES STATEMENT

```
CLEARSPRITES
```

Hides all 80 sprites.

```
CLEARSPRITES
```

COLLIDE FUNCTION

```
COLLIDE(x1, y1, w1, h1, x2, y2, w2, h2)
```

Returns TRUE if two rectangles overlap. Perfect for sprite collisions.

```
IF COLLIDE(px, py, 16, 16, ex, ey, 16, 16) THEN HitPlayer
```

12 Joypad input

Both joypad ports are read 60 times per second in the vertical blank interrupt. 3-button and 6-button pads are detected automatically.

BUTTON FUNCTION

```
BUTTON(mask [, pad])
```

TRUE while any of the given buttons is held on pad 1 (or pad 2).

```
IF BUTTON(BTN_LEFT) THEN x -= 2
IF BUTTON(BTN_A OR BTN_C) THEN Fire
```

PRESSED FUNCTION

```
PRESSED(mask [, pad])
```

TRUE only in the frame a button went down (it was not held at the previous VSYNC). Use it for jumps, menus and pause.

```
IF PRESSED(BTN_START) THEN Pause
```

JOY FUNCTION

```
JOY([pad])
```

All held buttons of pad 1 or 2 as a bit mask (see BTN_ constants).

```
j = JOY(1)
IF j AND BTN_UP THEN y -= 1
```

JOYNEW FUNCTION

```
JOYNEW([pad])
```

Buttons newly pressed since the last VSYNC, as a bit mask.

13 Maths

Integer maths in fixed point. Angles use 256 steps per circle.

ABS FUNCTION

```
ABS(x)
```

Absolute value.

```
d = ABS(px - ex)
```

SGN FUNCTION

```
SGN(x)
```

-1, 0 or 1 depending on the sign of x.

```
dir = SGN(targetX - x)
```

MIN FUNCTION

```
MIN(a, b)
```

The smaller of two values.

```
speed = MIN(speed + 1, 8)
```

MAX FUNCTION

```
MAX(a, b)
```

The larger of two values.

```
hp = MAX(hp - damage, 0)
```

CLAMP FUNCTION

```
CLAMP(value, low, high)
```

Limits a value to a range.

```
x = CLAMP(x, 0, 304)
```

RND FUNCTION

```
RND(n)
```

A random whole number from 0 to n-1.

```
x = RND(320)
IF RND(100) < 25 THEN SpawnBomb
```

SIN FUNCTION

```
SIN(angle)
```

Sine. The full circle is 256 steps (64 = quarter turn). Result -256..256 (fixed point, 256 = 1.0).

```
x = cx + (SIN(a) * radius) >> 8
```

COS FUNCTION

```
COS(angle)
```

Cosine, same units as SIN.

```
vx = COS(dir) * speed >> 8
```

ATAN2 FUNCTION

```
ATAN2(dy, dx)
```

The angle (0-255) of a direction. 0 = right, 64 = down, 128 = left, 192 = up.

```
a = ATAN2(py - ey, px - ex)   ' aim at the player
```

SQR FUNCTION

```
SQR(x)
```

Integer square root.

```
dist = SQR(dx * dx + dy * dy)
```

RANDOMIZE STATEMENT

```
RANDOMIZE seed
```

Starts the random number sequence from a seed. RANDOMIZE TIMER after a 'press start' gives a different game every time.

```
RANDOMIZE TIMER
```

Worked example: orbits and aiming

Twelve stars circle the centre using SIN and COS; a rock always points at the player using ATAN2.

examples/reference/orbit.bas

```
' SIN / COS / ATAN2: sprites orbiting and an "eye" following the player
SPRITES gfx = "sprites.png", 2, 2
CONST N = 12
DIM px, py, a, k
LOADTILES gfx
LOADPAL gfx, 1
FOR k = 0 TO N + 1
    SPRITEPAL k, 1
NEXT
px = 60 : py = 60
DO
    IF BUTTON(BTN_LEFT) THEN px -= 2
    IF BUTTON(BTN_RIGHT) THEN px += 2
    IF BUTTON(BTN_UP) THEN py -= 2
    IF BUTTON(BTN_DOWN) THEN py += 2
    SPRITE 0, px, py, gfx, 0
    a += 2
    FOR k = 1 TO N
        ' 12 stars on a circle of radius 70 around the centre
        SPRITE k, 152 + (COS(a + k * 256 / N) * 70 >> 8), 104 + (SIN(a + k * 256 / N) *
70 >> 8), gfx, 2
    NEXT
    ' a rock that slowly flies towards the player
    dir = ATAN2(py - 104, px - 152)
    PRINT AT 1, 1, "ANGLE TO PLAYER: "; PAD$(dir, 3)
    SPRITE N + 1, 152 + (COS(dir) * 30 >> 8), 104 + (SIN(dir) * 30 >> 8), gfx, 4
    VSYNC
LOOP
```



orbit.bas

14 Memory access

Direct memory access for experts. Work RAM is $FF0000-$FFFFFF.

PEEK FUNCTION

```
PEEK(address)
```

Reads a byte (0-255) from memory.

```
v = PEEK($FF0000)
```

PEEKW FUNCTION

```
PEEKW(address)
```

Reads a 16-bit word (address must be even).

PEEKL FUNCTION

```
PEEKL(address)
```

Reads a 32-bit long (address must be even).

POKE STATEMENT

```
POKE address, value
```

Writes a byte to memory. For experts - wrong addresses can crash the program.

POKEW STATEMENT

```
POKEW address, value
```

Writes a 16-bit word (even address).

POKEL STATEMENT

```
POKEL address, value
```

Writes a 32-bit long (even address).

15 Built-in constants

Predefined names. They cannot be redefined.

TRUE CONSTANT

```
TRUE = -1
```

Logical true (-1).

FALSE CONSTANT

```
FALSE = 0
```

Logical false (0).

PLANE_A CONSTANT

```
PLANE_A = 0
```

Foreground plane (text is drawn here by default).

PLANE_B CONSTANT

```
PLANE_B = 1
```

Background plane.

WINDOW CONSTANT

```
WINDOW = 2
```

The non-scrolling window plane (see WINDOWROWS).

BTN_UP CONSTANT

```
BTN_UP = 1
```

D-pad up.

BTN_DOWN CONSTANT

```
BTN_DOWN = 2
```

D-pad down.

BTN_LEFT CONSTANT

```
BTN_LEFT = 4
```

D-pad left.

BTN_RIGHT CONSTANT

```
BTN_RIGHT = 8
```

D-pad right.

BTN_B CONSTANT

```
BTN_B = 16
```

Button B.

BTN_C CONSTANT

```
BTN_C = 32
```

Button C.

BTN_A CONSTANT

```
BTN_A = 64
```

Button A.

BTN_START CONSTANT

```
BTN_START = 128
```

START button.

BTN_Z CONSTANT

```
BTN_Z = 256
```

Button Z (6-button pad).

BTN_Y CONSTANT

```
BTN_Y = 512
```

Button Y (6-button pad).

BTN_X CONSTANT

```
BTN_X = 1024
```

Button X (6-button pad).

BTN_MODE CONSTANT

```
BTN_MODE = 2048
```

MODE button (6-button pad).

HFLIP CONSTANT

```
HFLIP = 2048
```

Tile flag: mirror horizontally.

VFLIP CONSTANT

```
VFLIP = 4096
```

Tile flag: mirror vertically.

PRIORITY CONSTANT

```
PRIORITY = 32768
```

Tile flag: draw in front of low-priority sprites and tiles.

SCREEN_W CONSTANT

```
SCREEN_W = 320
```

Screen width in pixels.

SCREEN_H CONSTANT

```
SCREEN_H = 224
```

Screen height in pixels.

16 Music and sound effects

The sound driver runs on the 68000 in the vertical blank interrupt: 10 music channels (6 FM + 3 PSG tone + noise) and up to 4 simultaneous sound effects that temporarily take over a channel.

Channels

Name	Chip	Notes
`FM1` - `FM6`	Yamaha YM2612	4-operator FM, stereo panning. 8 octaves (o0-o7).
`PSG1` - `PSG3`	TI SN76489	Square waves, 16 volume steps; lowest note about A2 (110 Hz) - lower notes are clamped.
`NOISE`	SN76489	White (`w1` , default) or periodic (`w0`) noise; note picks the rate.

Timing

Durations are counted in ticks: a whole note is 96 ticks, a quarter 24, so lengths 1, 2, 3, 4, 6, 8, 12, 16, 24, 32, 48 and dotted values are exact. TEMPO is quarter notes per minute; at 150 BPM one tick is one frame. The driver corrects the tempo for 50 Hz PAL consoles, so music plays at the same speed everywhere. Sound effects have their own tempo (default 150).

MML command list

Command	Meaning
`c d e f g a b`	Notes. Add + or # for sharp, - for flat, then an optional length and dots: c4, f+8, b-2., e16
`r`	Rest (silence), with optional length: r4, r8.
`^`	Tie: lengthens the previous note: c4^16
`o0 .. o7`	Set the octave (o4 contains middle C... A = 440 Hz).
`> <`	One octave up / down.
`l<n>`	Default note length: l4 quarter notes, l8 eighths, l16 sixteenths. Dots allowed (l8.).
`v0 .. v15`	Volume (15 = loudest).
`@name`	Instrument (FM channels) or envelope (PSG/NOISE channels): @piano, @bass, @pluck ...
`t<bpm>`	Change tempo inside a channel.
`q1 .. q8`	Gate: how much of each note sounds (q8 = full, q4 = half, staccato).
`p0 / p1 / p2`	FM pan: centre / left / right.
`s<n>`	Pitch sweep per frame (s-20 falls, s12 rises): lasers, jumps, drums. s0 = off.
`w0 / w1`	NOISE channel: periodic (buzzy) / white noise. On NOISE, notes c-c# = high, d-d# = mid, e-f# = low, g-b follow PSG3's pitch.
`[ ... ]n`	Repeat the part in brackets n times (default 2). Brackets can be nested.

`L`	Loop point: when the channel reaches its end it jumps back here. Without L the channel plays once.
`| spaces`	Ignored - use them to make bars readable.

MML examples

```
"o4 l8 c d e f g4 g4"          ' C D E F (eighths), G G (quarters)
"o5 c4. d8 e2"                ' dotted quarter, eighth, half
"o3 l16 [c e g >c<]4"         ' arpeggio, 4 times
"@bass v13 o2 L c4 c4 g4 g4"  ' loop from L forever
"o4 c2^8 r8 q4 [c d e f]2"    ' tie, rest, staccato
"@laser o6 s-40 c8"           ' falling pitch sweep
```

Built-in FM instruments

Use with `@name` on FM channels. Index 0 (`piano`) is the default.

Name	Algorithm / feedback	Character
`piano`	2 / 5	acoustic-like piano
`epiano`	4 / 3	bell-like electric piano
`bass`	0 / 6	round FM bass
`slapbass`	3 / 5	bright slap bass
`synthbass`	0 / 7	buzzy synth bass
`lead`	4 / 6	detuned saw-like lead
`brass`	4 / 5	brass section, slower attack
`strings`	4 / 4	soft pad, slow attack and release
`organ`	7 / 0	drawbar organ (algorithm 7)
`flute`	4 / 2	breathy sine lead
`bell`	4 / 0	metallic bell / chime
`guitar`	0 / 7	distorted guitar
`sine`	7 / 0	pure sine wave
`square`	0 / 7	square-ish lead
`kick`	7 / 0	kick drum (use low notes, e.g. o2 c)
`snare`	0 / 7	noisy snare (high feedback)
`tom`	4 / 3	tom drum
`laser`	4 / 7	for effects with sweeps

Built-in PSG envelopes

Name	Attenuation per frame (0 = loud, 15 = silent)
`flat`	`0`

`pluck`	`0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15`
`decay`	`0, 0, 1, 1, 2, 2, 3, 3, 4, 4, 5, 5, 6, 6, 7, 7, 8, 8, 9, 9, 10, 10, 11, 11, 12, 12, 13, 13, 14, 14, 15`
`short`	`0, 2, 5, 9, 15`
`swell`	`12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0`
`hat`	`0, 4, 8, 12, 15`
`drum`	`0, 1, 3, 5, 7, 9, 11, 13, 15`
`soft`	`3`
`echo`	`0, 2, 4, 6, 8, 6, 8, 10, 12, 10, 12, 14, 15`

Designing FM instruments

A YM2612 voice has four operators (sine oscillators). The **algorithm** decides how they are connected; operators that reach the output are *carriers*, the others *modulators* that change the carriers' tone. Modulator TL sets brightness; carrier TL sets loudness (the driver adds the channel volume `v` to the carriers).

ALG	Connection	Carriers	Typical use
0	1 → 2 → 3 → 4	4	bass, distorted sounds
1	(1 + 2) → 3 → 4	4	bass, leads
2	(1 + (2 → 3)) → 4	4	pianos
3	((1 → 2) + 3) → 4	4	bass, snares
4	(1 → 2) + (3 → 4)	2, 4	brass, strings, e-piano
5	1 → (2 + 3 + 4)	2, 3, 4	organs, pads
6	(1 → 2) + 3 + 4	2, 3, 4	organs, chimes
7	1 + 2 + 3 + 4	all	additive organ, sine

Field	Range	Meaning
AR	0-31	attack rate (31 = instant)
DR	0-31	decay rate from peak down to SL
SR	0-31	sustain rate: decay while the key is held (0 = hold)
RR	0-15	release rate after key off
SL	0-15	sustain level (0 = full volume, 15 = silent)
TL	0-127	total level / attenuation (0 = loudest)
KS	0-3	key scaling: higher notes have faster envelopes
MUL	0-15	frequency multiple (0 = ½)
DT	-3..3	small detune
FEEDBACK	0-7	operator 1 self-modulation (higher = noisier)

Channel stealing

When a sound effect starts on a channel, the music on that channel is muted but keeps running silently. When the effect ends, the music channel's instrument is restored and it continues in time. If an effect starts while 4 are already playing, the oldest slot is reused. Choose channels for effects that the music uses least (PSG3, NOISE, FM6).

MUSIC KEYWORD

```
MUSIC name
    TEMPO bpm
    FM1 "mml..."
    PSG1 "mml..."
    NOISE "mml..."
END MUSIC
```

Defines a song. Each line gives the notes of one channel in MML text: FM1-FM6 are the six YM2612 FM voices, PSG1-PSG3 the three square-wave voices and NOISE the noise channel - all can play together. A channel can be continued on more lines. TEMPO is in beats per minute.

```
MUSIC theme
    TEMPO 140
    FM1  "@lead v12 o5 l8 L c d e g e d c4"
    FM2  "@bass v13 o2 l4 L c g a f"
    NOISE "@hat v9 l8 L [c c c c]2"
END MUSIC
```

SFX KEYWORD

```
SFX name
    [TEMPO bpm]
    PSG3 "mml..."
END SFX
```

Defines a sound effect in the same MML language. While an effect plays it borrows its channel(s) from the music, which continues afterwards. Up to 4 effects can play at once. Default SFX tempo is 150 (1 tick per frame).

```
SFX jump
    PSG3 "@short v14 o4 s12 l16 c"
END SFX

PLAYSFX jump
```

TEMPO KEYWORD

```
TEMPO bpm
```

Tempo of a MUSIC or SFX block in beats (quarter notes) per minute.

FM1 KEYWORD

```
FM1 "mml"
```

FM channel 1 (also FM2, FM3, FM4, FM5, FM6).

PSG1 KEYWORD

```
PSG1 "mml"
```

PSG square wave channel 1 (also PSG2, PSG3).

NOISE KEYWORD

```
NOISE "mml"
```

PSG noise channel - drums, explosions, hi-hats.

INSTRUMENT KEYWORD

```
INSTRUMENT name
    ALGORITHM 0-7
    FEEDBACK 0-7
    OP1 AR, DR, SR, RR, SL, TL, KS, MUL, DT
    OP2 ...
    OP3 ...
    OP4 ...
END INSTRUMENT

INSTRUMENT name = PRESET "brass"
```

Defines an FM instrument (a YM2612 patch) to use with @name in FM channels. Operator values: AR attack rate 0-31, DR decay rate 0-31, SR sustain rate 0-31, RR release rate 0-15, SL sustain level 0-15, TL total level (volume, 0 = loudest, 127 = silent), KS key scale 0-3, MUL frequency multiple 0-15, DT detune -3..3. Or copy a preset under a new name.

```
INSTRUMENT bell2
    ALGORITHM 4
    FEEDBACK 0
    OP1 31, 10, 4, 4, 5, 30, 1, 7, 0
    OP2 31, 6, 3, 4, 4, 6, 1, 1, 0
    OP3 31, 12, 4, 4, 5, 32, 1, 3, 3
    OP4 31, 6, 3, 4, 4, 10, 1, 1, 0
END INSTRUMENT
```

ENVELOPE KEYWORD

```
ENVELOPE name = v1, v2, v3, ...
```

Defines a PSG volume envelope for @name in PSG/NOISE channels: one attenuation value per frame (0 = loudest, 15 = silent). The last value is held.

```
ENVELOPE ping = 0, 0, 1, 2, 4, 6, 9, 12, 15
```

PLAYMUSIC STATEMENT

```
PLAYMUSIC song
```

Starts a song from the beginning (replacing the current one).

```
PLAYMUSIC theme
```

STOPMUSIC STATEMENT

```
STOPMUSIC
```

Stops the music.

PAUSEMUSIC STATEMENT

```
PAUSEMUSIC
```

Pauses the music (RESUMEMUSIC continues it).

RESUMEMUSIC STATEMENT

```
RESUMEMUSIC
```

Continues paused music.

MUSICVOLUME STATEMENT

```
MUSICVOLUME level
```

Music volume 0-15 (sound effects are not affected).

```
MUSICVOLUME 8
```

MUSICPLAYING FUNCTION

```
MUSICPLAYING()
```

TRUE while a song is playing (songs without a loop point L end by themselves).

```
PLAYMUSIC fanfare
DO : VSYNC : LOOP WHILE MUSICPLAYING()
```

PLAYSFX STATEMENT

```
PLAYSFX effect
```

Plays a sound effect.

```
PLAYSFX coin
```

STOPSFX STATEMENT

```
STOPSFX
```

Stops all sound effects.

Worked example: custom instruments and a jukebox

examples/reference/instruments.bas

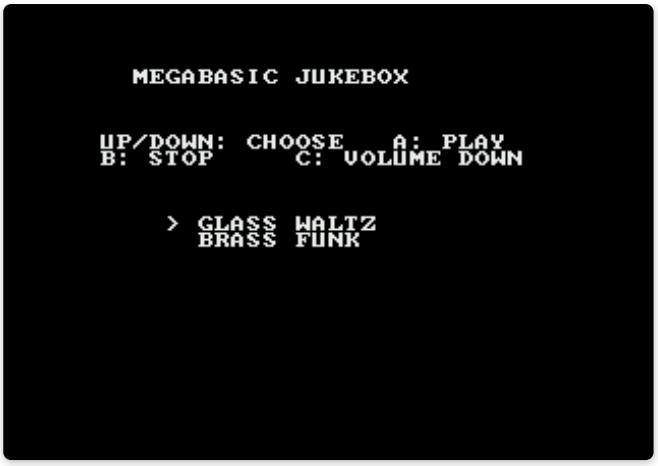
```
' Custom FM instrument, PSG envelope and a jukebox menu
INSTRUMENT glass
    ALGORITHM 4
    FEEDBACK 3
    '   AR  DR  SR  RR  SL  TL  KS MUL  DT
    OP1 31, 12,  4,  5,  6, 34,  1,  7,  0
    OP2 31,  5,  2,  5,  3,  4,  1,  1,  0
    OP3 31, 14,  4,  5,  6, 38,  1,  3,  2
    OP4 31,  5,  2,  5,  3,  8,  1,  1,  0
END INSTRUMENT

INSTRUMENT bigbrass = PRESET "brass"
ENVELOPE wah = 6, 4, 2, 0, 0, 1, 2, 3, 4, 5, 6, 7, 8

MUSIC song1
    TEMPO 110
    FM1  "@glass v13 o5 l8 L [e g b >e< b g]2 [d f+ a >d< a f+]2"
    FM2  "@bass v12 o2 l4 L e e e e d d d d"
    PSG1 "@wah v12 o4 l4 L e g f+ a"
END MUSIC

MUSIC song2
    TEMPO 150
    FM1  "@bigbrass v12 o4 l8 L c4 e g >c4< g e f4 a >c f4 c< a"
    FM2  "@slapbass v13 o2 l8 L [c c >c< c]2 [f f >f< f]2"
    FM3  "@organ v8 o4 l1 L e f"
    FM5  "@snare v11 o3 l4 L [r c]4"
    FM6  "@kick v14 o2 l4 L [c r]4"
    NOISE "@hat v8 l16 L [c c d c]8"
END MUSIC

DIM sel, vol = 15
PRINT AT 8, 4, "MEGABASIC JUKEBOX"
PRINT AT 6, 8, "UP/DOWN: CHOOSE   A: PLAY"
PRINT AT 6, 9, "B: STOP     C: VOLUME DOWN"
DO
    IF PRESSED(BTN_UP) THEN sel = 0
    IF PRESSED(BTN_DOWN) THEN sel = 1
    IF sel = 0 THEN PRINT AT 10, 13, "> GLASS WALTZ  " ELSE PRINT AT 10, 13, "  GLASS
WALTZ  "
    IF sel = 1 THEN PRINT AT 10, 14, "> BRASS FUNK   " ELSE PRINT AT 10, 14, "  BRASS
FUNK   "
    IF PRESSED(BTN_A) THEN
        IF sel = 0 THEN PLAYMUSIC song1 ELSE PLAYMUSIC song2
        vol = 15
        MUSICVOLUME vol
    END IF
    IF PRESSED(BTN_B) THEN STOPMUSIC
    IF PRESSED(BTN_C) THEN vol = MAX(vol - 3, 0) : MUSICVOLUME vol
    VSYNC
LOOP
```



instruments.bas

17 Built-in constants

Predefined names. They cannot be redefined.

Name	Value	Meaning
`TRUE`	-1	Logical true (-1).
`FALSE`	0	Logical false (0).
`PLANE_A`	0	Foreground plane (text is drawn here by default).
`PLANE_B`	1	Background plane.
`WINDOW`	2	The non-scrolling window plane (see WINDOWROWS).
`BTN_UP`	1	D-pad up.
`BTN_DOWN`	2	D-pad down.
`BTN_LEFT`	4	D-pad left.
`BTN_RIGHT`	8	D-pad right.
`BTN_B`	16	Button B.
`BTN_C`	32	Button C.
`BTN_A`	64	Button A.
`BTN_START`	128	START button.
`BTN_Z`	256	Button Z (6-button pad).
`BTN_Y`	512	Button Y (6-button pad).
`BTN_X`	1024	Button X (6-button pad).
`BTN_MODE`	2048	MODE button (6-button pad).
`HFLIP`	2048	Tile flag: mirror horizontally.
`VFLIP`	4096	Tile flag: mirror vertically.
`PRIORITY`	32768	Tile flag: draw in front of low-priority sprites and tiles.
`SCREEN_W`	320	Screen width in pixels.
`SCREEN_H`	224	Screen height in pixels.

18 Hardware, memory and limits

What the runtime sets up, where things live, and how far you can go.

VRAM address	Use
$0000-$B3FF	Tiles 0-1439. Tile 0 is blank; 1-1439 are for your graphics (LOADTILES allocates from 1 upward).
$B400-$BFFF	Font, tiles 1440-1535
$C000	Plane A name table (64 × 32)
$D000	Window name table
$E000	Plane B name table (64 × 32)
$F000	Sprite attribute table (80 sprites)
$FC00	Horizontal scroll table

RAM address	Use
$FF0000-$FF7FFF	Large arrays (over 256 bytes) - 32 KB
$FF8000-$FFEFFF	Runtime, sound driver, variables and small arrays (fast absolute short addressing) - about 28 KB
$FFF000-$FFFFFF	Stack (SUB/FUNCTION calls, locals, expression temporaries)

Limit	Value
Screen	320 × 224 pixels (H40, V28), 40 × 28 text cells
Numbers	32-bit signed integers
Single array	32,767 bytes; each dimension index below 32,768
Variables	about 28 KB (7,000 numbers) + 32 KB of arrays
Recursion depth	limited by the 4 KB stack (about 100 nested calls with a few locals)
Sprites	80 total, 20 per scan line, 1-4 × 1-4 tiles each
Colours	4 palettes x 16 (colour 0 transparent), 512 possible colours
Tiles for graphics	1439
Music	10 channels, 4 simultaneous effects
ROM size	up to 4 MB
Strings	text literals in PRINT and DATA (no string variables)

Performance

Measured on the emulated 7.67 MHz 68000 (one frame = 1/60 s):

Operation	Time
`FOR` loop with an addition, 10,000 iterations	9 frames

array read-modify-write, 10,000 times	19 frames
32-bit multiply + divide, 1,000 times	4 frames
`SPRITE` update, 1,000 times	4 frames
user FUNCTION call with 3 parameters, 1,000 times	3 frames
`COLLIDE`, 1,000 times	1 frame

Tip Speed tips: multiply/divide by powers of two become shifts automatically; `INC` / `+=` with small constants compile to one instruction; `BUTTON` / `PRESSED` with a constant pad read memory directly; MOD and general multiplication call a library routine.

19 How the compiler works

For the curious - and for anyone who wants to extend MegaBASIC.

1. **Lexer and parser** (`lexer.py`, `parser.py`) build a syntax tree. SUB/FUNCTION names are collected first, so procedures can be used before they are defined.
2. **Code generator** (`codegen.py`) resolves names, folds constant expressions, allocates variables and emits 68000 assembly. Expressions are evaluated into register d0; simple operands (constants, variables) are used directly as instruction operands. Conditions compile to compare-and-branch with short-circuit AND/OR. SUBs/FUNCTIONs use a `LINK`/`UNLK` stack frame on a6: parameters at positive, locals at negative offsets.
3. **Library** written in MegaBASIC itself (`builtins.py PRELUDE`): FADEIN, FADEOUT, MAPSCROLL. Only routines that are used are compiled in.
4. **Runtime** (`runtime/core.s`): reset/TMSS, VDP set-up, vertical blank interrupt, VSYNC with sprite DMA, text, tiles, maps, joypads, maths.
5. **Sound driver** (`runtime/sound.s`) and **MML compiler** (`sound.py`).
6. **Assets** (`assets.py`): pure-Python PNG decoder, palette quantiser, tile/sprite/map converters.
7. **Assembler** (`asm68k.py`): a multi-pass 68000 assembler with branch and address-size relaxation that writes the final ROM, header and checksum.

For example, this MegaBASIC code...

```
IF BUTTON(BTN_LEFT) THEN shipx -= 3
shipx = CLAMP(shipx, 0, 304)
```

...becomes this 68000 code (from `--asm`):

Generated assembly

```
    moveq #0,d0
    move.w rt_pad1,d0        ; held buttons of pad 1
    and.l #4,d0              ; BTN_LEFT
    beq __L52
    subq.l #3,g_SHIPX        ; shipx -= 3 : one instruction
__L52:
    move.l g_SHIPX,d0        ; CLAMP = MIN(MAX(v, 0), 304)
    moveq #0,d1
    cmp.l d1,d0
    bge.s __L54
    move.l d1,d0
__L54:
    move.l #304,d1
    cmp.l d1,d0
    ble.s __L53
    move.l d1,d0
__L53:
    move.l d0,g_SHIPX
```

Calling runtime routines

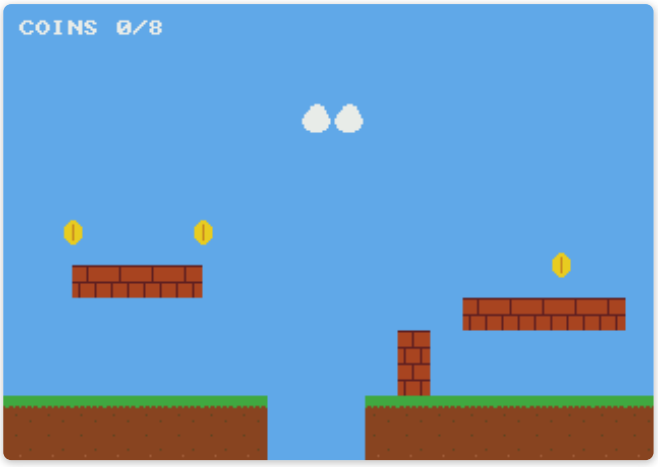
Built-in commands pass their arguments in d0-d7 (and a0 for asset addresses). Runtime routines may change d0-d7/a0-a3 and preserve a4-a6. User procedures receive their arguments on the stack (pushed right to left) and return values in d0.

20 Example programs

Complete programs included in the examples folder.

Platform game (examples/platformer)

A scrolling map, gravity, jumping, walls and ceilings tested with MAPTILE, coins as sprites positioned from DATA, music and effects.



platformer.bas

examples/platformer/platformer.bas

```
' ============================================================
'  PLATFORM DEMO - scrolling map, gravity, tile collisions
' ============================================================
TITLE "PLATFORM DEMO"

TILES ground = "tiles.png", 16          ' 16x16 metatiles
SPRITES hero = "hero.png", 2, 2         ' 4 walking frames
SPRITES items = "tiles.png", 2, 2       ' same picture as sprites: frame 4 = coin
MAP level = "level.csv", 16             ' 64 x 14 tiles made in Tiled

' tile numbers in the Tiled tileset (0 = empty)
CONST T_GRASS = 1, T_DIRT = 2, T_BRICK = 3

' physics in 1/16 pixel units
CONST GRAVITY = 5, JUMP_SPEED = -92, WALK = 22, MAX_FALL = 100
CONST NCOINS = 8

SFX jump
    PSG3 "@short v13 o4 s12 l16 c"
END SFX
SFX coin
    PSG3 "@pluck v14 o6 l32 b >e4"
END SFX
SFX fall
    PSG3 "@decay v14 o5 s-6 l2 c"
END SFX

MUSIC tune
    TEMPO 132
    FM1  "@square v10 o5 l8 L e g >c< g a f d f g e c e d4 g4"
    FM2  "@bass v13 o2 l8 L [c >c<]4 [f >f<]2 [g >g<]2"
    NOISE "@hat v8 l8 L [c d]8"
END MUSIC

DIM px, py, vx, vy, onGround, facing, camx, frame, coins
DIM coinX(NCOINS), coinY(NCOINS), coinGot(NCOINS) AS BYTE

' ---- is the map solid at pixel (x, y)? ----
FUNCTION Solid(x, y)
    LOCAL t
    t = MAPTILE(level, x >> 4, y >> 4)
    RETURN t = T_GRASS OR t = T_DIRT OR t = T_BRICK
END FUNCTION

SUB StartLevel()
    px = 32 * 16
    py = 160 * 16
    vx = 0
    vy = 0
    MAPRESET PLANE_B                    ' force a full redraw of the map
END SUB

SUB ReadCoins()
    RESTORE coindata
    FOR i = 1 TO NCOINS
        READ coinX(i), coinY(i)
        coinGot(i) = FALSE
    NEXT
END SUB
```

```
60
61  SUB MovePlayer()
62      LOCAL nx, ny, x, y
63      ' ---- walking ----
64      vx = 0
65      IF BUTTON(BTN_LEFT) THEN vx = -WALK : facing = TRUE
66      IF BUTTON(BTN_RIGHT) THEN vx = WALK : facing = FALSE
67      nx = px + vx
68      x = nx >> 4
69      y = py >> 4
70      IF vx > 0 THEN
71          IF Solid(x + 13, y + 2) OR Solid(x + 13, y + 14) THEN nx = px
72      ELSEIF vx < 0 THEN
73          IF Solid(x + 2, y + 2) OR Solid(x + 2, y + 14) THEN nx = px
74      END IF
75      px = MAX(0, nx)
76      ' ---- gravity and jumping ----
77      vy = MIN(vy + GRAVITY, MAX_FALL)
78      ny = py + vy
79      x = px >> 4
80      y = ny >> 4
81      onGround = FALSE
82      IF vy > 0 THEN
83          IF Solid(x + 3, y + 16) OR Solid(x + 12, y + 16) THEN
84              ny = (((y + 16) >> 4) * 16 - 16) << 4     ' stand exactly on the tile
85              vy = 0
86              onGround = TRUE
87          END IF
88      ELSEIF vy < 0 THEN
89          IF Solid(x + 3, y) OR Solid(x + 12, y) THEN
90              ny = py
91              vy = 0
92          END IF
93      END IF
94      py = ny
95      IF onGround AND PRESSED(BTN_A OR BTN_B OR BTN_C) THEN
96          vy = JUMP_SPEED
97          PLAYSFX jump
98      END IF
99      ' fell into a pit?
100     IF (py >> 4) > 232 THEN
101         PLAYSFX fall
102         WAIT 30
103         StartLevel
104     END IF
105 END SUB
106
107 SUB DrawWorld()
108     LOCAL sx, sy
109     x = px >> 4
110     camx = CLAMP(x - 152, 0, level.WIDTH * 16 - 320)
111     MAPSCROLL level, PLANE_B, ground, 0, camx, 0
112     ' player
113     frame += 1
114     IF vx <> 0 AND onGround THEN anim = (frame >> 2) AND 3 ELSE anim = 0
115     SPRITEFLIP 0, facing, FALSE
116     SPRITE 0, x - camx, py >> 4, hero, anim
117     ' coins are sprites (frame 4 of the items sheet)
118     FOR i = 1 TO NCOINS
119         IF coinGot(i) THEN
120             SPRITEOFF i
```

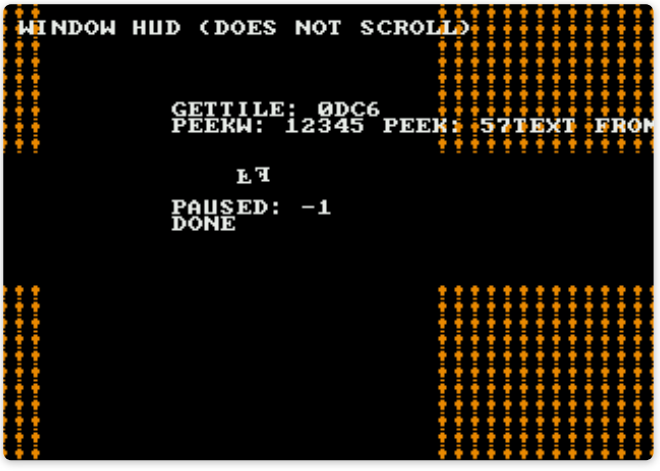
```
        ELSE
            sx = coinX(i) - camx
            sy = coinY(i)
            SPRITE i, sx, sy, items, 4
            IF COLLIDE(x + 2, py >> 4, 12, 16, coinX(i), coinY(i), 16, 16) THEN
                coinGot(i) = TRUE
                coins += 1
                PLAYSFX coin
            END IF
        END IF
    NEXT
    PRINT AT 1, 1, "COINS "; coins; "/"; NCOINS
END SUB

' ---------------- main ----------------
LOADTILES ground
LOADTILES hero
LOADTILES items
LOADPAL ground, 0
LOADPAL hero, 1
SETCOLOR 0, 0, RGB(100, 160, 255)
BACKDROP 0, 0
TEXTPAL 3
SPRITEPAL 0, 1
ReadCoins
StartLevel
PLAYMUSIC tune
DO
    MovePlayer
    DrawWorld
    VSYNC
LOOP

coindata:
DATA 120, 104,  184, 104,  360, 120,  432, 72
DATA 600, 104,  664, 56,  872, 120,  960, 104
```

Feature tour (examples/reference/features.bas)

Window plane HUD, FILLTILES, SETTILE flip flags, GETTILE, PEEK/POKE, text DATA, custom instrument and envelope, music pause and volume, independent plane scrolling.



features.bas

examples/reference/features.bas

```
INSTRUMENT myorgan
    ALGORITHM 7
    FEEDBACK 0
    '   AR DR SR RR SL TL KS MUL DT
    OP1 31, 0, 0, 8, 0, 20, 0, 1, 0
    OP2 31, 0, 0, 8, 0, 24, 0, 2, 0
    OP3 31, 0, 0, 8, 0, 28, 0, 3, 0
    OP4 31, 0, 0, 8, 0, 30, 0, 4, 0
END INSTRUMENT
INSTRUMENT horn = PRESET "brass"
ENVELOPE wobble = 0, 2, 4, 2, 0, 2, 4, 6, 8
MUSIC t
    TEMPO 120
    FM1 "@myorgan v12 L o4 c4 e4 g4 e4"
    FM2 "@horn v10 L o3 c1"
    PSG1 "@wobble v12 L o5 c2 g2"
END MUSIC
DIM v, w, hits
WINDOWROWS 3
TEXTPLANE WINDOW
PRINT AT 1, 1, "WINDOW HUD (DOES NOT SCROLL)"
TEXTPLANE PLANE_A
FILLTILES PLANE_B, 0, 4, 40, 24, 1440 + 33 - 32, 2          ' fill with '!' chars,
palette 2
SETCOLOR 2, 15, RGB(255, 128, 0)
SETTILE PLANE_A, 5, 10, 1440 + 70 - 32, 0, VFLIP            ' upside-down F
SETTILE PLANE_A, 6, 10, 1440 + 70 - 32, 0, HFLIP            ' mirrored F
w = GETTILE(PLANE_A, 6, 10)
PRINT AT 1, 6, "GETTILE: "; HEX$(w, 4)
POKEW $FF0100, 12345
PRINT AT 1, 7, "PEEKW: "; PEEKW($FF0100); " PEEK: "; PEEK($FF0101)
RESTORE msg
DO
    READ v
    IF v = 0 THEN EXIT DO
    PRINT CHR$(v);
LOOP
PRINT
PLAYMUSIC t
MUSICVOLUME 10
WAIT 30
PAUSEMUSIC
PRINT AT 1, 12, "PAUSED: "; MUSICPLAYING()
WAIT 30
RESUMEMUSIC
FOR y = 0 TO 300
    SCROLL PLANE_B, y, y / 2
    SCROLL PLANE_A, -y / 4, 0
    VSYNC
NEXT
PRINT AT 1, 13, "DONE"
DO : VSYNC : LOOP
msg:
DATA "TEXT FROM DATA!"
```

The complete game **STAR CATCHER** is explained line by line in chapter 28 of the Beginner's Guide.

21 Index

`:` Program structure
`ABS` Maths
`AND` Variables, arrays and data
`AS` Variables, arrays and data
`AT` Text output
`ATAN2` Maths
`BACKDROP` Colours and palettes
`.BASE` Graphics assets
`BGCOLOR` Colours and palettes
`BRIGHTNESS` Colours and palettes
`BTN_A` Built-in constants
`BTN_B` Built-in constants
`BTN_C` Built-in constants
`BTN_DOWN` Built-in constants
`BTN_LEFT` Built-in constants
`BTN_MODE` Built-in constants
`BTN_RIGHT` Built-in constants
`BTN_START` Built-in constants
`BTN_UP` Built-in constants
`BTN_X` Built-in constants
`BTN_Y` Built-in constants
`BTN_Z` Built-in constants
`BUTTON` Joypad input
`BYTE` Variables, arrays and data
`CALL` Program structure
`CASE` Decisions and loops
`CHR$` Text output
`CLAMP` Maths
`CLEARPLANE` Background planes and maps
`CLEARSPRITES` Sprites
`CLS` Text output
`COLLIDE` Sprites
`COLOR` Text output
`CONST` Program structure
`CONTINUE` Decisions and loops
`COS` Maths
`DATA` Variables, arrays and data
`DEC` Variables, arrays and data
`DIM` Variables, arrays and data
`DO` Decisions and loops
`DRAWMAP` Background planes and maps
`ELSE` Decisions and loops
`ELSEIF` Decisions and loops
`END` Program structure
`ENVELOPE` Music and sound effects
`EXIT` Decisions and loops
`FADEIN` Colours and palettes
`FADEOUT` Colours and palettes
`FALSE` Built-in constants
`FILLTILES` Background planes and maps
`FM1` Music and sound effects
`FOR` Decisions and loops
`.FRAMES` Graphics assets
`.FTILES` Graphics assets
`FUNCTION` Program structure
`GETCOLOR` Colours and palettes
`GETTILE` Background planes and maps
`GOTO` Program structure
`HEX$` Text output
`HFLIP` Built-in constants
`HIDESPRITE` Sprites
`IF` Decisions and loops
`INC` Variables, arrays and data
`INSTRUMENT` Music and sound effects
`IS` Decisions and loops
`JOY` Joypad input
`JOYNEW` Joypad input
`LET` Variables, arrays and data
`LOADPAL` Graphics assets
`LOADTILES` Graphics assets
`LOCAL` Program structure
`LOCATE` Text output
`LONG` Variables, arrays and data
`LOOP` Decisions and loops
`MAP` Graphics assets
`MAPHEIGHT` Background planes and maps
`MAPRESET` Background planes and maps
`MAPSCROLL` Background planes and maps
`MAPTILE` Background planes and maps
`MAPTILESIZE` Background planes and maps
`MAPWIDTH` Background planes and maps
`MAX` Maths
`MIN` Maths
`MOD` Variables, arrays and data
`MUSIC` Music and sound effects
`MUSICPLAYING` Music and sound effects
`MUSICVOLUME` Music and sound effects
`NEXT` Decisions and loops
`NOISE` Music and sound effects
`NOT` Variables, arrays and data
`OR` Variables, arrays and data
`PAD$` Text output
`PALETTE` Graphics assets
`PAUSEMUSIC` Music and sound effects
`PEEK` Memory access
`PEEKL` Memory access
`PEEKW` Memory access
`PLANE_A` Built-in constants

`PLANE_B` Built-in constants
`PLAYMUSIC` Music and sound effects
`PLAYSFX` Music and sound effects
`POKE` Memory access
`POKEL` Memory access
`POKEW` Memory access
`PRESSED` Joypad input
`PRINT` Text output
`PRIORITY` Built-in constants
`PSG1` Music and sound effects
`RANDOMIZE` Maths
`READ` Variables, arrays and data
`REM` Program structure
`REPEAT` Decisions and loops
`RESTORE` Variables, arrays and data
`RESUMEMUSIC` Music and sound effects
`RETURN` Program structure
`RGB` Colours and palettes
`RND` Maths
`SCREEN_H` Built-in constants
`SCREEN_W` Built-in constants
`SCROLL` Background planes and maps
`SELECT` Decisions and loops
`SETCOLOR` Colours and palettes
`SETTILE` Background planes and maps
`SFX` Music and sound effects
`SGN` Maths
`SHL` Variables, arrays and data
`SHR` Variables, arrays and data
`SIN` Maths
`SPRITE` Sprites
`SPRITEFLIP` Sprites
`SPRITEOFF` Sprites
`SPRITEPAL` Sprites
`SPRITEPRIORITY` Sprites
`SPRITES` Graphics assets
`SPRITESIZE` Sprites
`SQR` Maths
`STEP` Decisions and loops
`STOPMUSIC` Music and sound effects
`STOPSFX` Music and sound effects
`STR$` Text output
`SUB` Program structure
`SWAP` Variables, arrays and data
`TEMPO` Music and sound effects
`TEXTPAL` Text output
`TEXTPLANE` Text output
`THEN` Decisions and loops
`.TILES` Graphics assets
`TILES` Graphics assets
`.TILESIZE` Graphics assets
`TIMER` Timing
`TITLE` Program structure
`TO` Decisions and loops
`TRUE` Built-in constants
`UNTIL` Decisions and loops
`VFLIP` Built-in constants
`VSYNC` Timing
`.W / .H` Graphics assets
`WAIT` Timing
`WEND` Decisions and loops
`WHILE` Decisions and loops
`.WIDTH / .HEIGHT` Graphics assets
`WINDOW` Built-in constants
`WINDOWROWS` Text output
`WORD` Variables, arrays and data
`XOR` Variables, arrays and data
`_` Program structure